Operating Systems Lab

Class Assignment — Section A1

Priority-Scaled Round Robin

30 minutes. Fresh xv6 checkout, `CPUS := 1`.

The Idea

Replace the round-robin scheduler so that **how long a process's turn lasts** depends on its **priority** — not the order turns are handed out in.

- Turns still cycle through the process table in a fixed order, exactly like plain round robin.
- A process's turn lasts `BASE_QUANTUM * priority` ticks, instead of one fixed length for everyone.
- A turn still ends early if the process blocks (e.g. `sleep`) or exits before its quota is used up.

Nobody is ever skipped, so nobody starves — a low-priority process still gets a turn every cycle, just a shorter one.

Specification Parameters

Macro	Value	Meaning
`BASE_QUANTUM`	2	Ticks a priority-1 process gets per turn.
`MIN_PRIORITY`	1	Lowest priority a process may request.
`MAX_PRIORITY`	10	Highest priority a process may request.
`DEFAULT_PRIORITY`	3	Priority every process starts with.

System Calls

```
int setpriority(int priority);     int getpriority(void);
```

`setpriority` sets the calling process's priority; returns `0`, or `-1` (leaving/resetting the priority to `DEFAULT_PRIORITY`) if `priority` is outside `[MIN_PRIORITY, MAX_PRIORITY]`. `getpriority` returns the current value. A `fork()` child inherits its parent's *current* priority.

Logging

Toggleable log line, printed every time a process is picked for a *fresh* turn:

```
TURN: Process 5 (testloop) starts a new turn at priority 4 (quantum 8)
```

Test Program

Use the attached `testloop.c` as-is: save it to `user/testloop.c` and add `$U/_testloop` to `UPROGS` in the Makefile. Calling syntax: `testloop <iterations> <priority>`.

Sample Session

Three testloop processes, equal work, different priorities, launched one line at a time:

```
$ testloop 60 8 &
PID 4: starting 60 iterations at time 42, priority 8
$ testloop 60 4 &
PID 6: starting 60 iterations at time 90, priority 4
PID 4: finished at time 98
$ testloop 60 2 &
PID 8: starting 60 iterations at time 106, priority 2
PID 6: finished at time 152
PID 8: finished at time 181
```

Equal work, but priority 8 finishes first (98), well ahead of priority 4 (152) and priority 2 (181).

Submission: `git diff HEAD > ../<studentID>.patch`